

CACHING

Here is a concise, two-line summary for each layer of the caching architecture:

- **Browser Cache (Client-Side):** Stores static assets (like images and CSS) directly on the user's device. It eliminates network latency entirely by loading files locally on repeat visits.
- **CDN Cache (Content Delivery Network):** Geographically distributed servers that store static media physically closer to the user. It reduces network travel time and absorbs massive traffic spikes away from your origin servers.
- **Load Balancer Cache:** Sits at the network edge to store SSL sessions or identical, frequently requested API responses. It intercepts duplicate requests before they consume expensive backend application resources.
- **Application Cache (Local Memory):** In-memory storage (RAM) located directly within the specific application server processing the request. It holds immediate, temporary computations to provide lightning-fast access without network hops.
- **Distributed Cache (Redis / Memcached):** A shared, centralized cluster of high-speed memory accessible by all application servers. It acts as the primary shield for the database by storing the heavily requested "hot 20%" of data.
- **Database Buffer Pool:** The database's own internal RAM cache used to hold frequently queried data pages. It optimizes database performance by preventing slow, expensive physical disk reads.

Cache Hit: The data exists in the cache. Return it immediately.

Cache Miss: The data is not in the cache. Fetch from the source, optionally store in cache, then return.

If your database can handle 10,000 QPS, a 90% hit ratio means your system can effectively handle 100,000 QPS.

High Cache Value

Data Type	Why It Works
Read-heavy data	Same data requested many times
Expensive computations	Aggregations, joins, transformations
Slow data sources	External APIs, legacy systems
Stable data	Configuration, reference data

Poor Cache Candidates

Data Type	Why It Does Not Work
Write-heavy data	Cache invalidation overhead exceeds benefit
Unique requests	Each request needs different data
Large objects	Consume cache memory quickly
Time-sensitive data	Stale data is unacceptable

Cache invalidation is **the process of removing or updating stale, outdated data from a cache**

Consistency Approaches

Approach	How It Works	Trade-off
TTL-based	Data expires after a time period	Simple but allows staleness up to TTL
Invalidation	Explicitly remove/update cache on changes	Consistent but complex to implement
Write-through	Update cache and database together	Consistent but higher write latency
Eventual consistency	Accept temporary staleness	High performance but requires tolerance

The right approach depends on your consistency requirements. A product price being stale for 5 minutes might be acceptable. A bank account balance being stale for 5 seconds is not.

The 80/20 Rule

In most applications, 20% of the data serves 80% of the requests. Focus caching efforts on that hot 20%:

"Cache aggressively" (Cache Warming): For things you absolutely *know* will be hot (like the items featured on the homepage today), you might run a background job that proactively calculates and loads them into the cache before users even wake up.

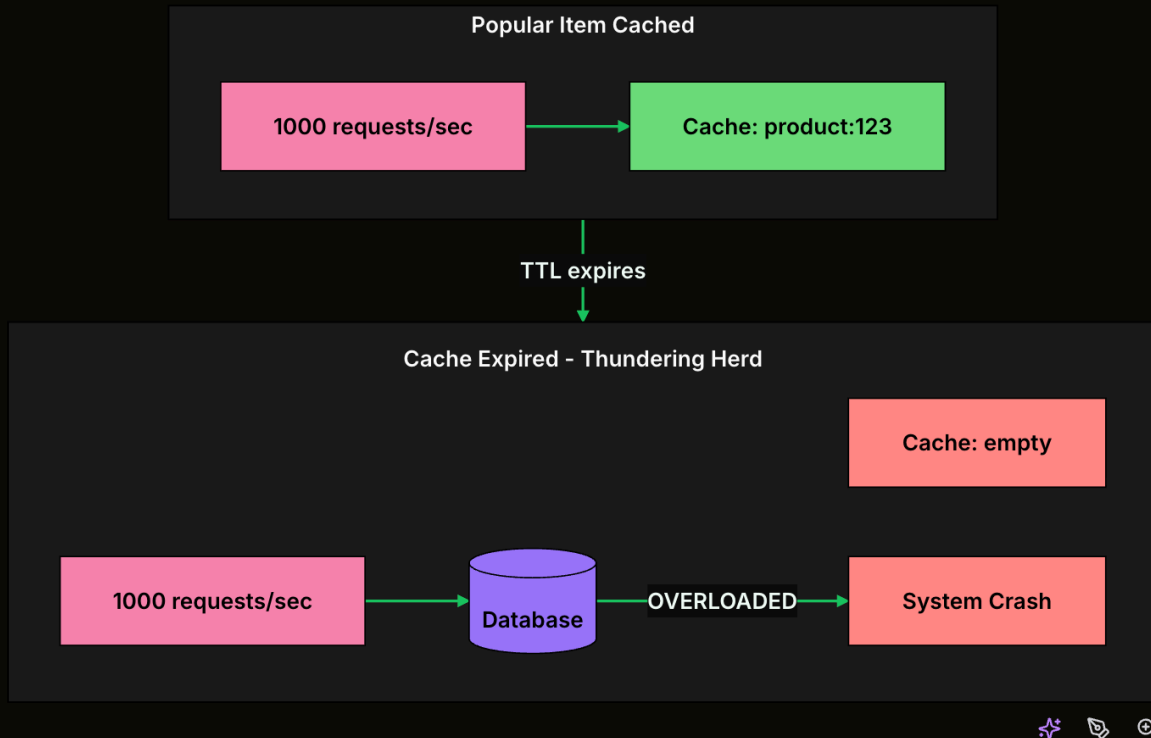
"On-demand with shorter TTL" (Cache-Aside): For everything else, you load it into the cache only *after* a user asks for it (on-demand), and you give it a short Time-To-Live (TTL, e.g., 5 minutes). If no one else asks for it within 5 minutes, it deletes itself to free up memory.

Caching Anti-Patterns

- **Cache Everything:** Wasting expensive memory by storing all data
- **Infinite TTL:** Keeping data forever guarantees stale information and creates fragile, error-prone manual invalidation requirements.
- **Cache as Primary Storage:** Treating temporary, non-durable memory as the source of truth, which risks permanent data loss.
- **The Thundering Herd:** Overwhelming the database with a sudden spike of simultaneous requests the exact moment a highly popular cached item expires.

The Thundering Herd

When a popular cache entry expires, many requests simultaneously hit the database:



Solutions include locking (only one request fetches), probabilistic early expiration, and background refresh.

Solution:

- allow only 1 req to hit db , rest all again from cache
- or cache warming: popular keys in cache will refresh periodically (so will not expire)

Eviction rate

The cache eviction rate is **the frequency or percentage of data items removed from a cache to make room for new data when the cache reaches capacity**. A high eviction rate indicates that the cache is too small or that it's experiencing high turnover, leading to lower hit rates and increased latency.

Cache in Distributed Systems

1. Consistency Across Nodes

- **The Problem:** When an underlying database record changes, the corresponding cache entry must be invalidated or updated across *all* cache nodes.
- **The Implication:** Because data takes time to travel over the network, replication introduces latency.

2. Data Partitioning (Consistent Hashing)

Large-scale applications cannot fit all their cached data onto a single machine, so the data must be partitioned (sharded) across multiple cache nodes.

- **The Solution:** Distributed caches use **Consistent Hashing**. This algorithm ensures that when a cache node is added or removed, only a very small fraction (roughly $1/N$) of the cached keys need to be moved to the new node, minimizing disruption to the system.

3. Failure Handling

Because network partitions and node crashes are inevitable in distributed systems, you must design how your application behaves when the cache layer becomes unavailable. The article outlines three primary strategies:

- **Fail Open:** : If the cache is unreachable, the system acts as if the cache doesn't exist. It "opens the gates" and lets all user requests flow directly to the database to get the data. It risks overloading the database (The Thundering Herd).
- **Fail Closed:** If the cache is unreachable, the system completely blocks the request. It does not attempt to contact the database at all. It immediately returns an error (like a 503 Service Unavailable or a friendly "Something went wrong" message) to the user. This approach is necessary when the cached data is absolutely critical to the request and cannot be retrieved fast enough or at all from the primary database.

Caching Strategies

1. Read Through

When the application requests data, it first looks in the cache.

If data is available (**cache hit**), it's returned to the application.

If the data is not available (**cache miss**), the cache itself is responsible for fetching the data from the database, storing it, and returning it to the application.

Read Through caching is best suited for **read-heavy applications** where data is accessed frequently but updated less often, such as content delivery systems (CDNs), social media feeds, or user profiles.

2. Cache Aside

The application first checks the cache for data. If the data exists in cache (**cache hit**), it's returned to the application.

If the data isn't found in cache (**cache miss**), the application retrieves it from the database (or the primary data store), then loads it into the cache for subsequent requests.

Cache Aside is perfect for systems where the **read-to-write** ratio is **high**, and data updates are infrequent. For example, in an e-commerce website, product data (like prices, descriptions, or stock status) is often read much more frequently than it's updated.

3. Write Through

This is a **synchronous process**, meaning both the cache and the database are updated as part of the same operation, ensuring that there is no delay in data propagation.

Write Through is ideal for **consistency-critical systems**, such as financial applications or online transaction processing systems, where the cache and database must always have the latest data.

4. Write Around

Write Around is a caching strategy where data is written directly to the database, bypassing the cache.

The cache is only updated when the data is requested later during a read operation, at which point the **Cache Aside** strategy is used to load the data into the cache.

Write Around caching is best used in **write-heavy systems** where data is frequently written or updated, but **not immediately or frequently read** such as logging systems.

5. Write Back

In the **Write Back** strategy, data is first written to the cache and then **asynchronously** written to the database at a later time.

However, with this approach, there is a risk of **data loss** if the cache fails before the data has been written to the database.

This can be mitigated by using persistent caching solutions like **Redis with AOF (Append Only File)**, which logs every write operation to disk, ensuring data durability even if the cache crashes.

Write Back caching is ideal for **write-heavy** scenarios where write operations need to be **fast** and **frequent**, but **immediate consistency** with the database is not critical, such as logging systems and social media feeds.

Caching strategies :

1. Least Recently Used (LRU)

Concept: Evicts the item that has not been accessed for the longest period. |

Usage: General-purpose caching (e.g., databases, web browsers).

Pros: Highly effective for recency-based patterns; keeps active data. | **Cons:** Requires extra memory and performance overhead to constantly track access order.

2. Least Frequently Used (LFU)

Concept: Evicts the item with the lowest overall access count. | **Usage:** CDNs and environments with predictable, long-term popular data.

Pros: Excellent at preserving consistently "hot" and popular data. | **Cons:** Fails to adapt quickly to changing trends and requires overhead for frequency counters.

3. First In, First Out (FIFO)

Concept: Evicts the oldest inserted item first, functioning like a standard queue. |

Usage: Simple buffers, sequential processing, or strict memory constraints.

Pros: Extremely simple to implement with zero access-tracking overhead. | **Cons:** Completely blind to data popularity, potentially evicting heavily used items.

4. Random Replacement (RR)

Concept: Evicts a randomly selected item when the cache reaches capacity. |

Usage: Systems needing ultra-low overhead or fallback eviction mechanisms.

Pros: Fast execution with absolutely zero tracking or memory overhead. | **Cons:** Unpredictable; runs the risk of evicting critical, high-demand data.

5. Most Recently Used (MRU)

Concept: Evicts the most recently accessed item first. | **Usage:** Looping sequential access patterns (like large database scans).

Pros: Prevents cache pollution when reading massive datasets where older data is needed again. | **Cons:** Performs very poorly in standard use cases where recent items are likely to be reused.

6. Time to Live (TTL)

Concept: Evicts items automatically once a predefined time duration expires. |

Usage: Session management, API responses, and time-sensitive dynamic content.

Pros: Guarantees data freshness and automatically cleans up stale memory. |

Cons: Difficult to choose the perfect time limit; might expire useful data prematurely.

7. Two-Tiered Caching

Concept: Combines a small, ultra-fast local cache with a larger, slower remote cache. | **Usage:** High-scale enterprise applications requiring both speed and massive storage (e.g., Local RAM + Redis).

Pros: Balances rapid local access latency with vast remote capacity. | **Cons:** High architectural complexity, requiring robust synchronization to keep tiers consistent.

When to use Redis (The Swiss Army Knife)

- **Complex Data Structures:** If your application needs to manipulate lists, sets, hashes, or bitmaps directly in the cache.
 - *Use Case:* Storing a user's shopping cart session as a Hash, or keeping track of unique daily visitors using HyperLogLog.
- **Leaderboards and Counting:** Redis has built-in "Sorted Sets" that make ranking incredibly fast.
 - *Use Case:* Real-time gaming leaderboards or trending topics on a social feed.
- **Pub/Sub Messaging:** Redis can act as a lightweight message broker.
 - *Use Case:* Real-time chat applications or triggering background jobs.
- **Data Persistence:** When you want your cache to survive a server restart (it periodically saves data to disk).
- **Geospatial Data:** Finding items within a certain radius.
 - *Use Case:* "Find drivers near me" in a ride-sharing app.

It supports data replication and persistence making it a popular choice for applications that require data durability and fault tolerance.

When to use Memcached (The Speed Demon)

Memcached does one thing and does it incredibly fast: it stores simple key-value pairs in memory.

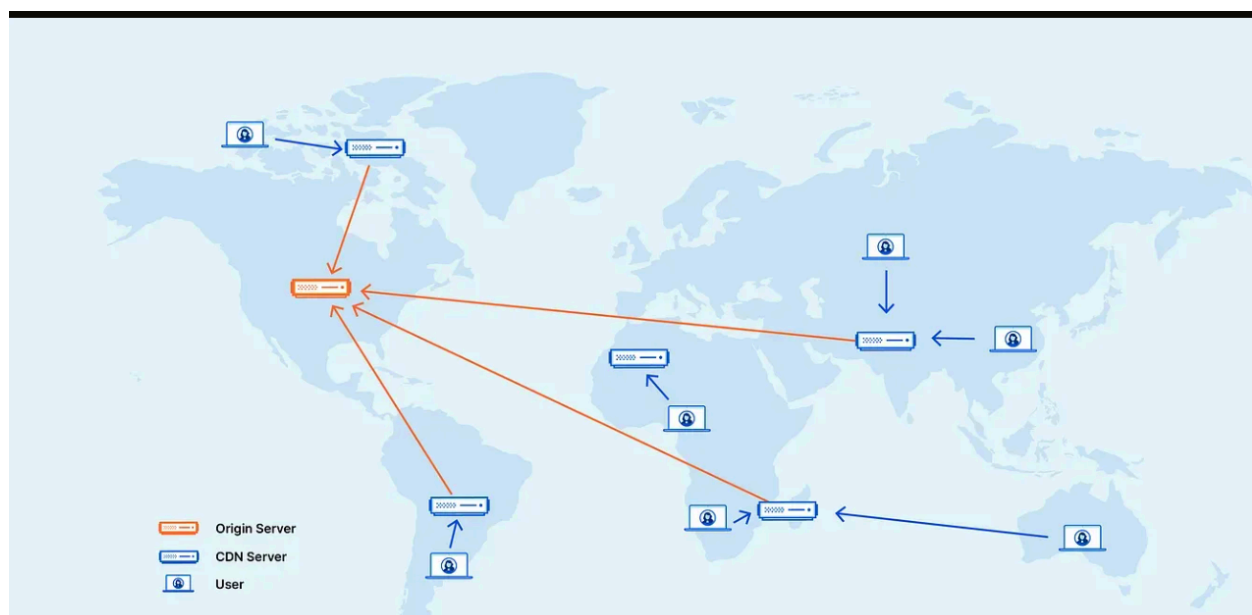
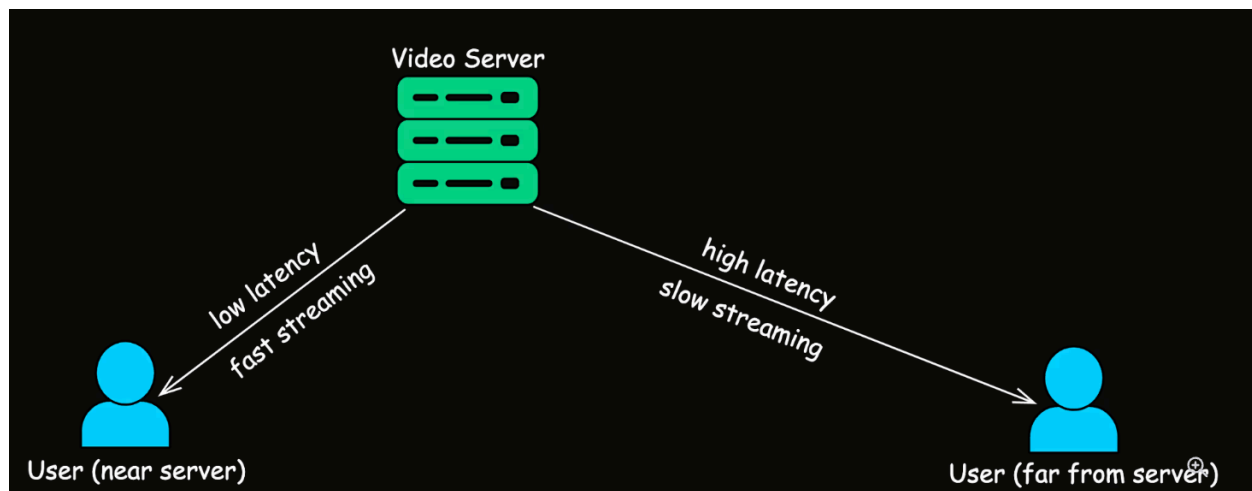
- **Pure HTML/Page Caching:** Storing pre-rendered blocks of HTML so your web servers don't have to generate them from scratch.
 - *Use Case:* Caching the header or footer of a high-traffic news website.
- **Simple Database Query Caching:** Storing the exact JSON results of expensive, read-heavy database queries.
- **Volatile Data Only:** Memcached has no persistence layer. This makes it ideal for use cases where data does not need to be stored permanently,

Amazon ElastiCache

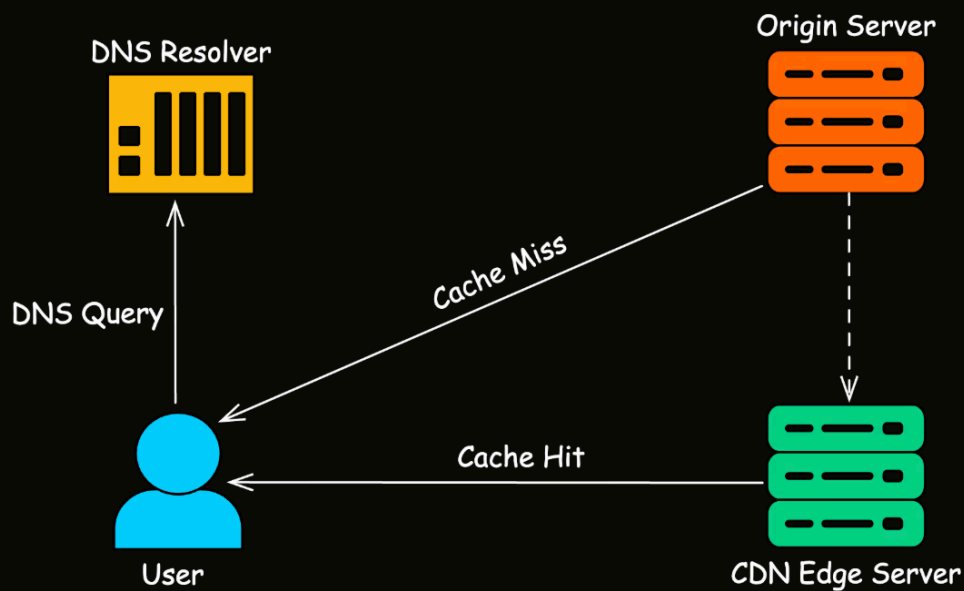
Amazon ElastiCache is a fully managed in-memory caching service provided by AWS. It supports both Redis and Memcached, allowing developers to choose the caching engine that best suits their needs.

Content Delivery Network (CDN)

Imagine you've built an app that serves **video content** to millions of users worldwide.



Here's a step-by-step breakdown of how a CDN works:



Many CDNs offer DDoS protection

4. Use Cases of CDNs

Accelerating Website Performance

- **CDNs cache static assets** (images, CSS, JavaScript) at **edge servers**, reducing the time required to fetch them from the origin.

Video Streaming & OTT Platforms

- **CDNs optimize video content delivery** by caching video files closer to users, minimizing buffering and latency.
- Supports **adaptive bitrate streaming** (ABR) to serve video based on the user's internet speed.
- **Example:** Netflix, YouTube, and Spotify use CDNs to serve videos and music in real-time to user.

Online Gaming

- Multiplayer **online games** require **low-latency** content delivery to ensure a smooth gaming experience.
- CDNs help distribute game updates, patches, and downloadable content (DLCs) faster.

Content & Media Distribution

- News websites and content platforms **deliver images, articles, and videos** through a CDN to handle large traffic spikes.

Software Distribution & Updates

- Operating system and software vendors use CDNs to **distribute large files, updates, and patches** quickly.
 - **Example:** Microsoft, Apple, and Google use CDNs for distributing Windows updates, macOS updates, and Android app downloads.
-

Popular CDN Providers

- **Need cheap/free DDoS protection and DNS management?** → Cloudflare
- **Need instant cache invalidation for real-time data?** → Fastly

If you are building a platform where content changes by the second (e.g., live sports updates, real-time financial dashboards, or breaking news sites), Fastly is famous for its instant cache purging capabilities (purging globally in milliseconds).

- **Already fully deployed on AWS/GCP/Azure?** → CloudFront / Google Cloud CDN / Azure CDN
- **A massive enterprise needing ultra-high bandwidth media delivery?** → Akamai
- **Massive, Global Scale:** If you are delivering high-bandwidth media (like a major streaming service launching a global movie premiere) or massive software downloads.
- **Enterprise and Compliance:** Large financial institutions, healthcare organizations, and government entities often choose Akamai for its enterprise-

grade SLAs, extensive compliance certifications, and deep threat intelligence network.
